

Ensuring Structured Outputs in Spring AI Applications

Practice 2 Transcript | Sep 29, 2026 18:57

Ethan Clarke 00:00

Good to see you. Our tests show the AI keeps returning plain text, which is hard to parse and slows down integration. What's your plan to make sure the outputs are structured and easy for our systems to consume?

You 02:37

The most effective way to eliminate parsing overhead is to transition our pipeline to strict schema enforcement. We can achieve consistently structured, machine-readable outputs through a three-pillar approach:

****1. Leverage Native Structured Outputs****

Instead of relying purely on prompt engineering to politely ask the model for JSON, we will utilize the LLM API's native Function Calling or Structured Output capabilities. By passing a strict, predefined JSON schema that maps exactly to what our downstream systems expect, we constrain the model to populate a data object rather than generating a conversational response.

****2. Implement a Strict Validation Layer****

We will introduce a validation middleware—using a library like Pydantic for Python—between the AI model and our internal systems. Every AI response will be intercepted and checked against our exact schema definitions. If an output is missing required fields, includes unexpected keys, or uses the wrong data types, it gets caught immediately before it can break any downstream integrations.

****3. Automated Self-Correction Loops****

In the rare event that the model hallucinates a malformed structure, the validation layer won't just fail. Instead, it will automatically catch the parser error and feed it back to the AI in a fast, invisible retry loop, instructing it to fix the specific formatting mistake.

This approach guarantees that our downstream applications only ever receive clean, validated data objects, completely removing the need for fragile string-parsing logic on our end.

Ethan Clarke 02:41

That's a solid plan—really well-structured. Leveraging the LLM's native structured output capabilities is a smart move, especially if they support JSON schema enforcement. The validation middleware is also a great way to catch formatting issues early. One thing I'd ask is how you plan to design the retry logic for the self-correction loops. What kind of prompt adjustments or error details will you provide to guide the model in fixing mistakes?

Ensuring Structured Outputs in Spring AI Applications

You 03:42

The retry logic needs to be highly specific and tightly constrained to prevent runaway API costs or latency spikes. We will design the self-correction loop around targeted error injection and strict circuit breakers.

****1. Granular Error Injection****

When the validation layer catches an error, we won't just prompt the model to "try again." We will feed the exact parser exception back into the context window. The correction payload will include:

* **The malformed output:** The exact string or JSON the model just generated.

* **The specific parser error:** The raw validation failure (e.g., `ValidationError: 1 validation error for Response -> confidence\_score: Input should be a valid integer, got a string`).

* **The schema reminder:** The specific portion of the schema that was violated.

****2. The Correction Prompt Structure****

We will use an automated, system-level message appended to the conversation history designed to strip away conversational behavior. The prompt will look like this:

> **"Your previous output failed validation. Do not apologize or explain what went wrong. Return ONLY the corrected JSON object. Error details: [Insert Error]. Malformed payload: [Insert Payload]."**

****3. Circuit Breakers (Max Retries)****

To protect system latency, the retry loop will be hard-capped at two attempts. If an LLM cannot format its output correctly after being handed the exact error twice, the issue is likely a fundamental misalignment in the original prompt or schema complexity, not a transient generation glitch.

****4. Graceful Degradation and Telemetry****

If the model exhausts its retries, the system will catch the final exception and route the original query and malformed response to a dead-letter queue. This serves two purposes:

* **System Stability:** Downstream apps receive a clean HTTP 422 (Unprocessable Entity) or a safe default fallback, rather than crashing on a null pointer or broken string.

* **Continuous Improvement:** We will use the dead-letter queue as a targeted telemetry dataset. If we see a specific field (like a nested date string) repeatedly triggering the retry loop, it tells us exactly where we need to refine our base schema descriptions or add a few-shot example to the primary prompt.

Ethan Clarke 03:46

I love the level of detail here—this is really thought through. Feeding back the exact error and malformed payload as context is a fantastic idea for guiding the model toward precise corrections. The retry cap is another smart move to balance reliability with system performance. One thing to consider: how will you handle edge cases where the schema itself might need tweaking? For example, if repeated failures reveal ambiguity in field definitions, what's your plan for rapid schema updates while maintaining consistency across the pipeline?

Ensuring Structured Outputs in Spring AI Applications

You 05:01

Schema evolution is one of the most critical challenges in production AI, as changing the extraction rules can easily break downstream systems. To handle ambiguity and enable rapid updates without sacrificing stability, we need to treat our schemas exactly like API contracts.

Here is how we will manage the lifecycle of our schemas:

****1. Centralized Schema Registry and Strict Versioning****

We will implement a schema registry where every JSON schema is version-controlled (e.g., ``v1.0``, ``v1.1``, ``v2.0``). Downstream applications will explicitly request the version they are built to consume.

* ****Non-breaking changes**** (like adding an optional field) will bump the minor version.

* ****Breaking changes**** (like renaming a field, changing a data type, or adding a new required field) will bump the major version.

This ensures that while the AI team iterates rapidly on extraction models, downstream engineering teams have a stable, predictable contract until they are ready to migrate.

****2. Decoupling Schema Definitions from Application Code****

Schemas will not be hardcoded into the core backend services. Instead, they will live as separate configuration artifacts in their own repository. This allows the AI team to push schema updates through a lightweight CI/CD pipeline independently of the main engineering release cycle, enabling rapid deployment of fixes.

****3. Leveraging Schema Descriptions as Prompt Engineering****

When the dead-letter queue reveals ambiguity (e.g., the model keeps returning ``"Tuesday"`` instead of ``"2026-10-06"``), we often don't need to change the data structure itself. Because most LLM structured output frameworks support the ``description`` parameter within the JSON schema, we can update the description (e.g., changing "The date of the event" to "The date of the event, strictly in ISO-8601 format"). This instantly fixes the model's behavior without requiring any changes to the downstream parsing logic.

****4. Shadow Testing for Schema Updates****

Before pushing a schema tweak to production, we will deploy it in "shadow mode." We will take a sample of recent user queries—specifically those that previously ended up in the dead-letter queue—and run them against the updated schema in an offline test environment. If the new schema successfully resolves the validation errors without introducing regressions, it gets promoted to production.

By separating the schema definition from the application logic and enforcing strict versioning, we create a firewall between AI experimentation and system stability. The AI team gets the agility to fix extraction bugs in minutes, and the downstream teams get the guarantee that their endpoints will never suddenly change shape.

Ethan Clarke 05:05

Absolutely brilliant! Treating schemas like API contracts and maintaining strict versioning is exactly the kind of discipline needed to avoid chaos in production. I especially like the use of ``description`` in schemas for prompt fine-tuning—it's a lightweight but highly effective fix. Shadow testing is another top-notch idea for catching edge cases before they hit production. With this setup, you're creating an ecosystem that's both flexible for AI updates and stable for downstream systems. Anything else you're considering to make this even more bulletproof?

End of session